HUSTRONIX OS · IMPLEMENTATION REPORT

Database, Consistency, and Anti-Hallucination

Author: Hustronix OS
Date: 13 August 2026
Source: docs/database-consistency-and-context.md

Database, Consistency, and Anti-Hallucination Report

Status: Implementation report (code-backed)
Date: 13 August 2026
Audience: Founders, operators, and engineers who need one place that explains *where truth lives*, *how it stays consistent*, and *why the LLM context window is not the memory*.

Related: `nango-to-signals-architecture.md` · `architecture-enforcement.md` · `agents-and-slack-signals.md` · `entity-model.md` · `belief-engine.md`

How to read this document

If you want…	Go to
The one-sentence rule	§1 Thesis
What SQLite actually stores	§2 Storage model · §3 Schema map
How Slack/Gmail/GitHub become rows	§4 Ingest seam
Why two Slack retries don't create two truths	§5 Consistency guarantees
Why the auditor is not "the AI checking itself"	§6 Auditor
Durable memory vs chat history	§7 Memory & Company Brain
Why context window does not get to be the store	§8 Working set vs retrieval
Exact prompt caps (cost-saver defaults)	§9 Snapshot + cost saver
How "what's active?" avoids the model entirely	§10 L0 routing
What still can go wrong	§11 Known gaps
How to verify this in a running system	§12 Verification checklist

---

1. Thesis

The LLM is not the memory. SQLite is.

Hustronix is an operating system for **decisions and execution**. External events land in a **ledger**. Deterministic rules (the **auditor**) enforce capacity, shadow work, and conflict. Chat is an

interface into that ledger — not a second source of truth.

If you try to keep “all required data” inside the model’s context window:

1. History is truncated (by design; cost-saver mode is **on by default**).
2. The model will invent ids, vows, and task states that are not in the prompt.
3. Two turns will disagree because neither turn re-read the database.

The product rule is therefore:

> **Ledger + auditor = source of truth. LLM proposes and explains. Retrieval re-reads SQLite every turn. Mutations that matter become rows.**

“In memory” in this architecture means: **the SQLite file is durable on disk, the Node process holds an in-process `better-sqlite3` connection (pages cached in RAM), and each agent turn injects a small, deterministic snapshot plus on-demand tool reads.** It does **not** mean stuffing the org into the prompt.

---

2. Storage model

2.1 One database, in-process

Item	Value
Engine	SQLite via `better-sqlite3`
Class	`HustronixDB` in `apps/api/src/database.ts`
Default path	`./data/hustronix.db` (`SQLITE_PATH`)
Access pattern	Synchronous prepared statements in the API process
Why this shape	Ingest + auditor + snapshot are on the hot path; a remote DB round-trip would make “re-read before every LLM turn” expensive

Reads and writes are **transactional and local**. Persistence is the file. RAM only caches pages. Restarting the API does not lose ledger state.

There is no separate “AI memory database.” Chat, signals, decisions, executions, memory facts, FTS indexes, and embeddings all live in this file (plus additive V7–V9 security / entity / belief / reconciliation schemas).

2.2 What is *not* the store

System	Role	Must not become
Nango (or first-party OAuth)	Auth, token refresh, sync, provider webhooks	Signal store, Company Brain, or business schema
Chat `messages`	UX transcript	Org truth
LLM context window	Disposable working set	Durable memory
Embeddings (`entity_embeddings`)	Optional rerank after FTS	Source of truth
Nango record cache	Ephemeral fetch buffer	Second ledger

Invariant from the Nango contract: **Nango failure must never invalidate Hustronix evidence.** The permanent seam is `ingestExternal()` .

2.3 Three ledgers (not one dump)

```
external_signals     evidence from the world (Slack, Gmail, GitHub, …)
        │
        │  optional explicit link
        ▼
execution_ledger     tasks under a decision (ACTION / KILL / VERIFY / BLOCK)
        │
        │  FK decision_id
        ▼
decision_ledger      strategic vows (DRAFT → ACTIVE → COMPLETED / REVERSED / …)
```

Signals never auto-create an ACTIVE decision. Chain: signal → (optional) execution link → human/gated decision commit.

Supporting tables (not “the ledger,” but required for consistency):

- `audit_log` / `ledger_events` — traceability and signed event stream
- `external_event_dedup` / `slack_event_receipts` — ingest idempotency
- `messages` — chat archive (searchable; not canonical)
- `memory_records` + `memory_conflicts` + provenance — gated facts
- `signals_fts` / `executions_fts` / `decisions_fts` — FTS5 retrieval
- `entities` / `evidence_nodes` / `evidence_edges` — Company Brain graph
- `settings` keys `core_memory.*` — small prompt projections (org profile, world model, system prompt)

3. Schema map (what consistency is *about*)

3.1 `decision_ledger` — strategic commitments

Column (core)	Meaning
`id`	UUID
`user_id` / `org_id`	Tenant
`fingerprint`	SHA-256 of vow + expected outcomes + reasoning — **unique**, dedupes identical creates
`vow`	The commitment statement
`expected_outcomes`	JSON metrics/targets
`constraints`	JSON “NO list”
`reversal_conditions`	When to abandon
`confidence`	Structural / epistemic confidence
`status`	`DRAFT` | `ACTIVE` | `UNDER_REVIEW` | `COMPLETED` | `REVERSED`
`decision_reasoning`	Institutional reasoning JSON (v3 + server-attached `epistemic_derivation`)

Hard gate: activating a decision (`transitionDecisionState` → `ACTIVE`) refuses if another ACTIVE decision exists for the org. Audit action: `DECISION_CONFLICT_BLOCKED`. DRAFT does not count.

3.2 `execution_ledger` — enforcement graph

Column (core)	Meaning
`task_id`	UUID
`decision_id`	FK → `decision_ledger`
`owner`	Person / function
`type`	`ACTION` | `KILL` | `VERIFY` | `BLOCK`
`priority` / `weight`	Ordering and capacity cost
`status`	`PENDING` | `ACTIVE` | `DONE` | `KILLED` | `BLOCKED`

Column (core)	Meaning
`dependencies`	JSON task ids
`success_condition` / `failure_condition`	Deterministic checks
`blocked_reasons`	JSON reasons

Killing a decision cascades: pending/active/blocked tasks under it become `KILLED`.

3.3 `external_signals` — evidence ledger

Column (core)	Meaning
`id`	UUID
`user_id`	Tenant owner of the signal
`source`	`GITHUB` | `SLACK` | `JIRA` | `MICROSOFT` | `GOOGLE` | `GMAIL` | `NOTION` | `LINEAR` | `MANUAL`
`source_subtype`	Granular event name (e.g. `microsoft.outlook.mail.created`)
`content`	Human-readable evidence line
`owner`	Actor string (`slack:U123`, …)
`external_id`	Provider idempotency key
`ingest_meta`	JSON (includes `integration_runtime: "nango"` on Nango path)
`state`	`UNVERIFIED` | `GRACE` | `LINKED` | `KILLED`
`linked_execution_id`	FK → task when linked
`grace_expires_at`	Auditor countdown for unlinked shadow work
Interpretation cols	`intent`, `risk`, `urgency`, `op_classification`, `related_decision_id`, …

Evidence vs interpretation (non-negotiable):

Axis	Examples	Mutability
Source identity	`source`, `external_id`, `connection`, `source_subtype`	Immutable
Source event	`content`, `detected_at` / `occurred_at`, `owner`, `ingest_meta`	Immutable evidence

Axis	Examples	Mutability
Interpretation	`intent`, `risk`, `signal_type`, confidence, op class	Mutable; reprocessable
Governance	`state`, `trust_state`, `resolution_state`, `execution_link`	Mutable via explicit APIs / auditor

Detectors may be wrong. **Re-run intelligence; do not rewrite `content` / `external_id`.** Relink: `POST /api/intelligence/relink`.

Target progressive states (stashed in `ingest_meta` until full migration):

- `processing_state`: `RECEIVED` → `PROCESSING` → `PROCESSED` | `FAILED`
- `actionability_state`: `UNTRIAGED` → `RELEVANT` | `DISMISSED` | `LINKED`
- `resolution_state`: `ACTIVE` | `SUPERSEDED` | `RESOLVED`

Until migrated, `UNVERIFIED` / `LINKED` / `KILLED` remain the operational enum.

3.4 Search indexes (retrieval, not truth)

On insert/update, the DB layer maintains FTS5 virtual tables:

- `signals_fts` — content, owner, source
- `executions_fts` — success/failure conditions
- `decisions_fts` — vow, outcomes, constraints

`hybridSearch` = FTS candidates → optional embedding rerank (`entity_embeddings`).
Embeddings are **not** mandatory on the ingest hot path.

---

4. Ingest seam (how the world becomes rows)

Canonical path (Nango, approved):

```
provider webhook
    → verify signature
    → ACK 200 immediately
    → async dispatcher: listRecords (cursor)
    → ProviderAdapter → EvidenceEnvelope
    → envelopeToIngestRecord
    → ingestExternal() / ingestExternalBatch()
    → insertExternalSignal (UNVERIFIED)
    → auditor.onWrite({ type: "signal" })
         ├─ grace window
         └─ processSignalFireAndForget (detect → enrich → optional auto-link)
```

Permanent Hustronix API: `apps/api/src/integrations/ingestRouter.ts`.

4.1 `ingestExternal` responsibilities (one place)

1. **Tenant resolution** — explicit `userId`, else Slack team → `oauth_connections` / `nango_connections`, else drop (no silent cross-tenant write).
2. **Source bucket** — provider string → schema CHECK enum (`SLACK`, `GITHUB`, …).
3. **Idempotency** — `tryInsertExternalEventId(source, externalId, tenantKey)` into `external_event_dedup`. Duplicate → return existing `signalId`, `duplicate: true`.
4. **Schema-stable insert** — `content` trimmed, `ingest_meta` JSON, FTS row.
5. **Auditor hook** — not the LLM.

What ingest is **not:** a queue, a scheduler, a knowledge graph, or a decision creator.

Transitional: first-party Slack Events API (`slackEvents.ts` + `slack_event_receipts`) stays live until the Nango Slack (or Google) path is trusted in production. Same destination table.

4.2 Linking is a separate contract

Ingest only inserts `UNVERIFIED`. Becoming work is explicit:

Step	Mechanism
Triage	`GET /api/execution/signals/by-state`, Signals UI
Link	`POST /api/execution/link` `{ signalId, taskId }` → `LINKED` + `linked_execution_id`
Kill	`POST /api/execution/signals/kill` or auditor after grace

Auto-link (intelligence) may call the **same** link path when policy says so. It must not invent a parallel write.

---

5. Consistency guarantees (implemented)

These are **database and auditor rules**, not prompt instructions.

Guarantee	Where	What happens on violation
One ingest seam	`ingestExternal`	New providers add an adapter, not a second insert path
Webhook idempotency	`external_event_dedup` PK `(source, external_id, tenant_key)`	Second delivery is a no-op
Slack event dedup (legacy)	`slack_event_receipts`	Slack retries do not double-insert
Decision semantic dedup	`decision_ledger.fingerprint` UNIQUE	Identical vow+outcomes+reasoning returns existing id
At most one ACTIVE decision per org	`transitionDecisionState`	Throw + `DECISION_CONFLICT_BLOCKED`
Tenant isolation	`user_id` / `org_id` on reads	API + agent queries scoped; `system` automation does not get cross-tenant signals
Signal ≠ ACTIVE decision	Product + ingest	No ingest path writes `decision_ledger` as ACTIVE
Evidence immutable / interpretation reprocessable	Intelligence + schema	Enrich updates interpretation cols; not `external_id` / `content` identity
Execution FK integrity	`execution_ledger.decision_id` → `decision_ledger`	Orphan tasks cannot exist
Cascade kill	decision → `KILLED`	Child tasks pending/active/blocked → `KILLED`
Memory claim gate	`storeMemoryFact` + `validateAiClaim`	AI-sourced facts rejected or UNVERIFIED until review
Memory contradiction	`memory_conflicts`	Open conflicts recorded; trust metrics degrade
Content trust	`ContentTrustPipeline`	Non-SAFE content blocked from LLM + memory write
Prompt injection	classifier in `routeUserQuery`	High-risk input can skip LLM entirely

Guarantee	Where	What happens on violation
Auditor after mutation	`setLedgerAuditorHook` / `onWrite`	Shadow work, capacity, NO-list, collapse
Cron safety net	`*/5 * * * *` in bootstrap	Catches missed hooks + time-based grace
Signed ledger events	`ledger_events` + `LEDGER_SIGNING_KEY`	Tamper-evident event stream (when key set)

Nango replaceability: deleting a Nango connection row must not delete `external_signals`. Pilot test (`ops/scripts/nango-slack-pilot.test.ts`) asserts evidence survives connection deletion.

---

6. Auditor (enforcement that does not chat)

`apps/api/src/auditor.ts`

Trigger	Behavior
`onWrite({ type: "signal" })`	Init/check grace for that signal; fire-and-forget `processSignal`
`onWrite({ type: "decision" | "execution" | "signal_link" })`	Scoped evaluation (capacity, NO-list, structural stress)
`evaluateAll()` / cron every 5 minutes	Full sweep if a code path forgot the hook, or grace expired offline

Grace (unlinked `UNVERIFIED` signals) is a **deterministic countdown** (`grace_expires_at`). Slack default is +2 hours. Expired unlinked signals → `KILLED` (shadow work), with audit `SIGNAL_KILLED`.

The auditor **never** uses the LLM to decide whether a rule fired. Intelligence (detect/enrich/auto-link) is advisory and writes back through the same DB APIs, which re-enter the auditor.

This is the product's answer to "the model might hallucinate a kill/link": **kills and capacity enforcement are SQL + time, not prose.**

---

7. Memory & Company Brain (durable, gated)

Chat history is **not** organizational memory. Durable knowledge is promoted into typed stores.

7.1 `memory_records`

Field	Role
`tenant_id`	Isolation
`source_type`	`user_chat` | `agent_tool` | `slack` | `email` | `meeting` | `manual` | `system`
`content` + `content_hash`	Fact text + dedup hash
`verification_state`	`UNVERIFIED` | `VERIFIED` | `SUPERSEDED` | `REVOKED` | `ARCHIVED`
`confidence`	0–1
`block`	Optional projection key (`user_profile` , `persona` , org core memory)
provenance	`memory_provenance_events` + `memory_evidence` links

Write path for the agent tool `remember_fact`:

1. LLM output firewall
2. Content trust pipeline (`allowMemoryWrite`)
3. Claim validation (`validateAiClaim`) for AI/system-sourced facts
4. Insert `memory_records`
5. Contradiction scan vs last 50 live peers → `memory_conflicts` if score ≥ 0.65
6. Index as evidence graph node
7. `syncCoreMemoryProjection` — small `settings.core_memory.<block>` string for prompts

Human-verified input may skip claim validation. Agent-sourced facts may not.

7.2 Core memory projections (what the prompt actually sees)

Stored as `settings` keys `core_memory.<block>` (not writable via generic settings PATCH):

Block	Use
`system_prompt`	Optional tenant override of coordinator system prompt
`organization`	Company identity, ICP, north star (onboarding digest)

Block	Use
`world_model`	JSON hypotheses / confidence / regime (capped in prompt)

These are **projections**, not a second ledger. If projection and `memory_records` diverge, records + provenance win.

7.3 Weekly compression (why we delete context on purpose)

`apps/api/src/state/memory-compression.ts` — scheduled Sun 03:00 via bootstrap.

```
recent user messages (200)
    → heuristic fact candidates → memory_records (UNVERIFIED)
    → belief candidates → beliefs
    → garbage collection (archive / merge)
```

This is how old chat survives **without** keeping infinite history in the prompt. Compression is lossy by design; the ledger (decisions/executions/signals) is not compressed away.

7.4 Company Brain (derived, not ingested)

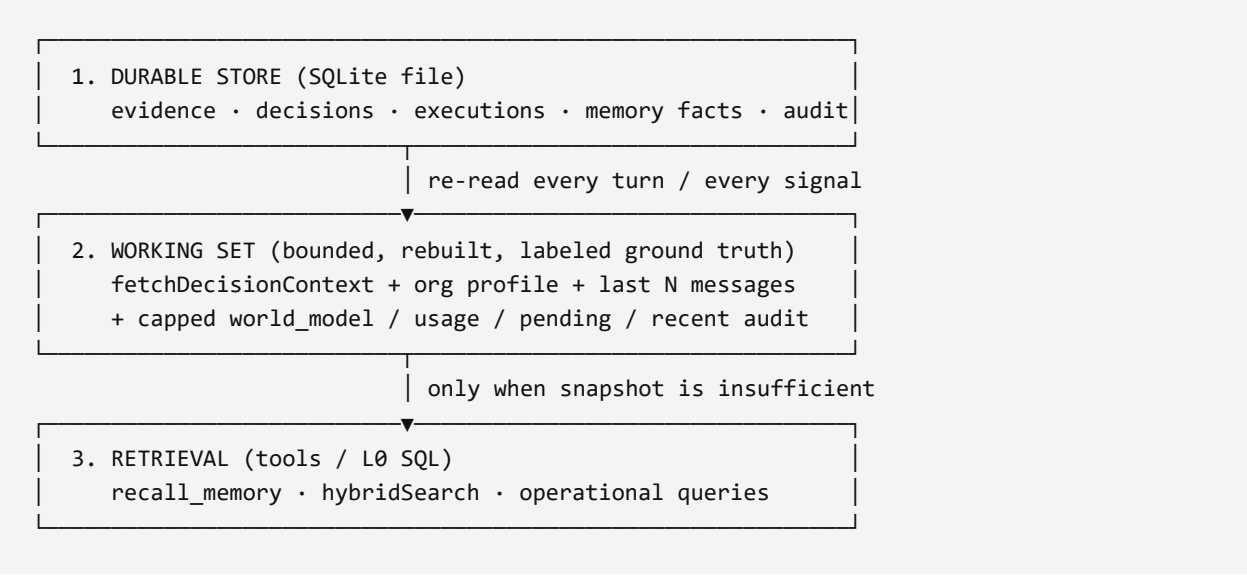
Entities (`company` , `team` , `decision` , `task` , `person` , …) and evidence edges are **derived from Hustronix evidence**, never from Nango's cache or provider APIs directly. See `entity-model.md` .

Organization state is a **projection** over that subgraph, optionally injected into the supervisor prompt (`ORG_STATE_ENABLED` , default on).

---

8. Working set vs retrieval (anti-hallucination architecture)

Three layers. Mixing them up is how hallucination starts.



8.1 Rules the coordinator is instructed to follow

From `buildSupervisorPrompt()` in `apps/api/src/agent.ts`:

1. **Decision context block is canonical for ledger facts.** Do not invent decisions, executions, or signals not listed there.
2. If the user asks about ledger items **not in that block**, say so (then use tools).
3. Organization profile, when present, is canonical for company identity — do not claim “I have no company context” while that block is populated.
4. `log_strategic_decision` writes **DRAFT** only; activation is gated server-side (causal validation, one ACTIVE, etc.).

Prompt rules without DB gates would be theater. The snapshot + L0 + tool reads + auditor are what make the rules real.

8.2 Per-turn snapshot (`fetchDecisionContext`)

`apps/api/src/intelligence/contextEngine.ts`

Pure SQLite read. **No LLM.** Called on:

- every coordinator turn
- every signal intelligence run (same slice the auto-linker uses)

Returns at most:

- one ACTIVE decision (vow, NO-list, reversal, confidence)
- live executions (capped)

- recent signals (capped, content truncated)

Rendered as a stable, positional block:

```
## DECISION CONTEXT (ledger snapshot — ground truth)
### ACTIVE DECISION (id=…)
VOW: …
EXPECTED_OUTCOMES: …
CONSTRAINTS (NO list): …
### LIVE EXECUTIONS (N total, showing K)
### RECENT SIGNALS (showing K)
## END CONTEXT
```

Same snapshot → chat prompt **and** auto-linker. Chat and linking cannot "see different truths" for the active surface.

8.3 Chat history is a transcript, not the org

`messages` table stores every user/assistant line. Coordinator loads only:

- last `CHAT_HISTORY_MESSAGES` rows (default **8** in cost-saver, **12** otherwise)
- each body capped at `CHAT_MESSAGE_CHAR_CAP` (default **1200** / **1600**)

Older chat is still on disk. The model does not see it unless `recall_memory` (LIKE search, last 10 hits) or compression has promoted a fact.

8.4 On-demand retrieval

Need	Mechanism	Source of truth?
"What did we say about X last month?"	`recall_memory` → `messages` LIKE	Transcript only
Semantic / operational search	`hybridSearch` (FTS → embed rerank)	Ledger rows
Drift / overload / related signals	`operationalQuery.ts`	Ledger SQL
"What's active / blocked / doctrine?"	**L0** — no LLM	Ledger / org state / doctrine
Org identity	`core_memory.organization` + org-state block	Projection of settings + state

---

9. Snapshot + cost-saver caps (exact defaults)

Cost-saver is **default on**. Disable with `LLM_COST_SAVER=0`.

Knob	Env	Cost-saver default	Full default	Max
Tool/LLM rounds per turn	`MAX_ITERATIONS`	3	4	20
History messages	`CHAT_HISTORY_MESSAGES`	8	12	80
Chars per history message	`CHAT_MESSAGE_CHAR_CAP`	1200	1600	32000
Signals fetched for context	`DECISION_CONTEXT_SIGNAL_LIMIT`	6	8	24
Executions shown in prompt	`DECISION_CONTEXT_MAX_EXECUTIONS`	6	10	50
Signals shown in prompt	`DECISION_CONTEXT_MAX_SIGNALS`	4	6	16
Vow chars	`DECISION_CONTEXT_MAX_VOW_CHARS`	360	480	4000
Expected-outcomes chars	`DECISION_CONTEXT_MAX_OUTCOMES_CHARS`	400	560	8000
Signal snippet chars	`DECISION_CONTEXT_SIGNAL_SNIPPET_CHARS`	100	120	500
Org context chars	`ORG_CONTEXT_PROMPT_MAX_CHARS`	2800	4000	12000
Supervisor JSON blobs	`SUPERVISOR_JSON_BLOB_MAX_CHARS`	550	850	32000
Recent audit outcomes	`SUPERVISOR_RECENT_OUTCOMES`	3	4	20
Tool result chars	`COORDINATOR_TOOL_RESULT_MAX_CHARS`	2200	3000	32000

These caps are **intentional**. They force retrieval. Raising them to “fit the whole org in the prompt” would increase hallucination surface and cost, not reduce it.

After tool results return, the coordinator may trim the previous assistant message **in memory only** (next LLM iteration) so the working set stays bounded.

---

10. L0 routing (SQL answers without a model)

`apps/api/src/reasoning/router.ts` → `cost-optimizer.ts`

Before any LLM call:

1. Prompt-injection classifier — may **block** LLM entirely.
2. `tryAnswerWithoutLlm` — if the question matches operational patterns, answer from SQLite/graph and **never call the model**.

User intent (regex)	L0 source
Active decision	Organization state `activeDecisions`
Blocked tasks	`execution_ledger` status `BLOCKED`
Capacity / overload	`findOverloadedOwners` / owner capacity
Doctrine	`listDoctrineRules`
What changed / diff	org state version diff
Entity neighbors / impact	entity store + evidence graph
Drift	`findDriftSignals`

This is the strongest anti-hallucination path: **the model cannot misquote a vow it never saw.**

Higher levels (L1–L5) still get a `contextBlock` from `buildContextForQuery` when LLM is used. High-risk mutation language (`commit`, `kill all`, …) raises dual-control.

---

11. Known gaps (honest)

Documented in `architecture-enforcement.md`; still true as of this report.

Gap	Risk	Intended fix
Coordinator tools `log_strategic_decision` and `transition_execution` write DB **directly**	Agent-as-controller; demo-convenient, not the long-term operating model	Propose → intent queue → validated HTTP/enforcement service → commit
Auto-link can apply without a human when policy allows	Wrong link if detector is wrong	Keep same `link` API; tighten `link-policy`; interpretation remains reprocessable

Gap	Risk	Intended fix
`recall_memory` is LIKE on `messages`, not hybrid search over memory facts	Weak archival recall	Point recall at `memory_records` + FTS/hybrid
Weekly compression heuristics are naive (keyword lines)	Low-quality UNVERIFIED facts	Keep UNVERIFIED + claim gate; improve extractors later
`user_id` vs `org_id` split-brain on some legacy rows	Tenant edge cases	V9 tenant-scope work (partial)
First-party OAuth still parallel to Nango	Two ingest islands until per-provider prune	Keep until Nango path proven per provider
Prompt snapshot is capped	Model can miss older signals/tasks not in the window	Correct: use tools / L0; do not uncap blindly

None of these gaps change the thesis: **do not move source of truth into the context window to paper over them.**

---

12. Verification checklist

Use this when asking “is required data consistent, and is the model grounded?”

Storage & ingest

- ☐ API log shows `[DB] Connected to:` the expected `SQLITE_PATH`.
- ☐ Slack/Nango sync produces rows in `external_signals` with stable `external_id`.
- ☐ Replay the same webhook / `POST /api/nango/sync/process` → `duplicate: true`, row count unchanged.
- ☐ Nango connection row deleted → signal rows **remain** (`integration_runtime = "nango"` in `ingest_meta`).
- ☐ `npm run test:nango-slack-pilot` (from `apps/api`) passes.

Ledger gates

- ☐ Two `ACTIVE` commits for one org: second fails with `DECISION_CONFLICT_BLOCKED`.
- ☐ Duplicate `log_strategic_decision` payload: same `fingerprint` → same decision id.
- ☐ Unlinked Slack signal past grace → `KILLED` + audit `SIGNAL_KILLED` (auditor or wait ≤ 5 min cron).

- [ ] `POST /api/execution/link` sets `state=LINKED` and `linked_execution_id`.

Anti-hallucination / context

- [ ] Ask “what’s the active decision?” → response path **L0** (no LLM) and matches `decision_ledger`.
- [ ] Ask about a task **not** in the snapshot window → coordinator says it isn’t in context and/or calls `recall_memory` / search — does not invent a `task_id`.
- [ ] After `log_strategic_decision`, next turn’s snapshot includes the new DRAFT/pending surface (or explicit “none ACTIVE” if not committed).
- [ ] `remember_fact` with injected/malicious content → blocked by trust/claim validation; no `memory_records` row (or conflict recorded).
- [ ] Cost-saver on: prompt sizes stay within the table in §9 (inspect coordinator logs / token usage).

Memory

- [ ] `GET /api/memory/facts/:id/explain` returns provenance (who, when, source, evidence, contradictions).
- [ ] Open `memory_conflicts` visible in trust metrics.
- [ ] Weekly compression audit `MEMORY_COMPRESSION_RUN` appears after schedule (or manual `runWeeklyMemoryCompression`).

---

13. Worked example (one Slack message)

1. Slack (or Nango `Message` sync) delivers an event.
2. Signature verified; webhook ACKs.
3. `ingestExternal` resolves tenant, dedups `external_id`, inserts `external_signals` `UNVERIFIED`.
4. Auditor sets `grace_expires_at`; `processSignal` classifies/enriches **interpretation** columns.
5. UI / Mission Control lists the signal. Human (or gated auto-link) calls `POST /api/execution/link`.
6. User asks chat: “What’s active?” → **L0** reads org state / ledger, returns the vow — no model.
7. User asks: “Should we kill shadow work on this thread?” → coordinator sees the signal in `fetchDecisionContext` (if in window) or retrieves it; recommends LINK or KILL per rules; **auditor** still KILLs on grace if nobody linked.
8. User commits a new strategy → `log_strategic_decision` writes **DRAFT**; `POST /api/decisions/commit` hits `transitionDecisionState`; second ACTIVE is impossible.

9. Next chat turn **re-reads** SQLite. The vow in the prompt is the row, not last week's prose.

Hallucination failure mode this prevents: the model "remembering" a different active vow than `decision_ledger.status='ACTIVE'`.

Hallucination failure mode this does **not** fully prevent: the model summarizing a truncated signal snippet incorrectly. Mitigation: short snippets + "not in context" rule + `hybridSearch` / open the Signals UI. Interpretation can be re-run.

---

14. Code & doc index

Area	Path
SQLite schema + ledger APIs	`apps/api/src/database.ts`
Ingest seam	`apps/api/src/integrations/ingestRouter.ts`
Nango dispatcher	`apps/api/src/integrations/nango/dispatcher.ts`
Evidence envelope	`apps/api/src/integrations/ingest/`
Adapters	`apps/api/src/integrations/adapters/`
Context snapshot	`apps/api/src/intelligence/contextEngine.ts`
Intelligence orchestrator	`apps/api/src/intelligence/index.ts`
Hybrid search	`apps/api/src/intelligence/hybridSearch.ts`
Coordinator + supervisor prompt	`apps/api/src/agent.ts`
L0 / routing	`apps/api/src/reasoning/router.ts`, `cost-optimizer.ts`
Auditor	`apps/api/src/auditor.ts`
Memory records / conflicts	`apps/api/src/security/memory-store.ts`
Memory facts + claim gate	`apps/api/src/security/memory-fact.ts`, `claim-validation.ts`
Compression	`apps/api/src/state/memory-compression.ts`
Prompt / history caps	`apps/api/src/config.ts` (`config.agent`)
Recall / remember tools	`apps/api/src/tools/recall-memory.ts`, `remember-fact.ts`
Security schema (memory, graph)	`apps/api/src/security/security-schema.ts`
Nango architecture	`docs/nango-to-signals-architecture.md`

Area	Path
Enforcement vs LLM	`docs/architecture-enforcement.md`
Agents + Slack Events	`docs/agents-and-slack-signals.md`

---

15. One-page operating contract

1. **SQLite is truth.** Chat is a transcript. Nango is plumbing.
2. **Re-fetch, don't remember.** Every agent turn rebuilds `fetchDecisionContext` from disk.
3. **If it isn't a row, it isn't a decision / signal / task.** Tool writes create rows; prose does not.
4. **Prefer L0** for "what is the state?"
5. **Idempotent ingest + one ACTIVE + auditor** keep the ledger internally consistent.
6. **Evidence is immutable; interpretation is re-runnable.**
7. **Compress old chat into gated facts.** Do not grow infinite context.
8. **Caps are a feature.** Retrieval exists because the window is small.
9. **Auditor, not the model, kills shadow work and blocks conflicting ACTIVE decisions.**
0. **Do not "fix hallucination" by pasting the whole database into the prompt.** That recreates the problem this system is built to avoid.